

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)
FORMAL LANGUAGES AND AUTOMATA THEORY (CS602PC)
PROJECT REPORT

TITLE OF PROJECT: System Validation Engine: Turing Machine Design for String Equality and DFA-Based Intrusion Detection

Submitted By:

Name: S GANESH CHARY , D YASASWINI

Roll No: 23261A0552 , 23261A0516

Branch: CSE

Year: 2025-2026

Submitted To: Ms S RENUKA Assistant Professor

Department of Computer Science and Engineering

MGIT, Hyderabad

Academic Year:2025-2026

CERTIFICATE

This is to certify that the project entitled **System Validation Engine: Turing Machine Design for String Equality and DFA-Based Intrusion Detection** is a bonafide work carried out by **S GANESH CHARY (23261A0552), D YASHAWINI(23261A0516)** in partial fulfillment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering.

Faculty Guide Signature: _____

Name: Ms S Renuka

Designation: Assistant Professor

Head of the Department Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to our faculty guide **Ms S Renuka** for their valuable guidance. I also thank the Head of the Department and college management for their support in completing this project.

INDEX

1. Introduction
2. Problem Statement
3. Objective
4. Theory
5. Design / Diagram
6. Example
7. Result
8. Conclusion
9. References

INTRODUCTION

The System Validation Engine is designed using the concept of **Turing Machines (TM)** from Formal Languages and Automata Theory (FLAT). A Turing Machine is a powerful computational model capable of performing complex symbolic manipulations on an infinite tape. In this project, it is applied to solve a fundamental verification problem: checking whether two input strings separated by the symbol “#” are identical.

String comparison is an important operation in many computational systems, especially in data validation, compiler design, and security applications. Unlike simple pattern matching techniques, a Turing Machine provides a step-by-step formal mechanism to process and verify strings systematically using state transitions and tape operations.

In this System Validation Engine, the Turing Machine reads input strings of the form **w#w**, where the left and right substrings are compared symbol by symbol. The machine marks matched symbols and continues the comparison until all characters are verified. If all corresponding symbols match, the machine accepts the input; otherwise, it rejects it.

This project demonstrates the practical application of theoretical computation models in solving real-world validation problems and highlights the importance of automata theory in system design and verification processes.

PROBLEM STATEMENT

The problem addressed in this System Validation Engine is the design of a Turing Machine that can determine whether two strings separated by the symbol “#” are identical.

Given an input string of the form:

$$w \# w$$

the system must verify whether the substring before “#” is exactly the same as the substring after “#”.

Traditional string comparison methods rely on high-level programming constructs, but this project aims to solve the same problem using a formal computational model. The challenge is to design a Turing Machine that can:

- Read symbols from both sides of the separator “#”
- Compare them one by one
- Mark matched symbols
- Detect mismatches
- Reach an accept or reject state accordingly

OBJECTIVE

The objectives of the proposed **Automata-Based Multi-Layer Authentication System Using DFA and Turing Machine** are as follows:

- To design and implement a secure authentication system using concepts from **Formal Languages and Automata Theory (FLAT)**.
- To develop a **Deterministic Finite Automaton (DFA)** for validating user login credentials through state transitions.
- To design a **Turing Machine (TM)** for processing and verifying One-Time Passwords (OTP).
- To implement a **Multi-Layer Authentication (MLA)** framework consisting of credential verification, OTP verification, and behavioral biometric verification.
- To demonstrate the practical application of **DFA** and **Turing Machine** concepts in real-world cybersecurity and authentication systems.

THEORY / MODEL USED

The System Validation Engine is developed using the concepts of **Formal Languages and Automata Theory (FLAT)**, specifically based on the computational model of a **Turing Machine (TM)**. The Turing Machine is a powerful theoretical model introduced by Alan Turing, capable of simulating any algorithmic computation using an infinite tape, a tape head, and a set of states.

In this project, the Turing Machine is used to solve the problem of **string equality checking**, where two strings are separated by the symbol “#”. The machine processes the input step by step, compares corresponding symbols from both sides, and determines whether they are identical.

Turing Machine (TM) Model

A Turing Machine is formally defined as:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Where:

- $Q \rightarrow$ Finite set of states
- Σ (**Sigma**) $\rightarrow$ Input alphabet

- Γ (**Gamma**) $\rightarrow$ Tape alphabet (includes input symbols + special markers)
- δ (**Delta**) $\rightarrow$ Transition function that defines state changes
- $q_0 \rightarrow$ Initial state
- $B \rightarrow$ Blank symbol
- $F \rightarrow$ Set of final (accepting) states

DESIGN / DIAGRAM

The proposed **System Validation Engine** is designed using a **Turing Machine (TM)** model to perform **string equality checking for two strings separated by the symbol “#”**. The system follows a structured **mark-and-compare approach**, where each symbol from the left and right substrings is matched sequentially using tape operations and state transitions.

The input string is processed on an infinite tape, and the Turing Machine uses a finite control unit to guide transitions between states. The system ensures that both strings are identical by marking matched symbols and rejecting any mismatch. The overall system architecture and working flow are illustrated below.

Fig. 1. Overall System Architecture

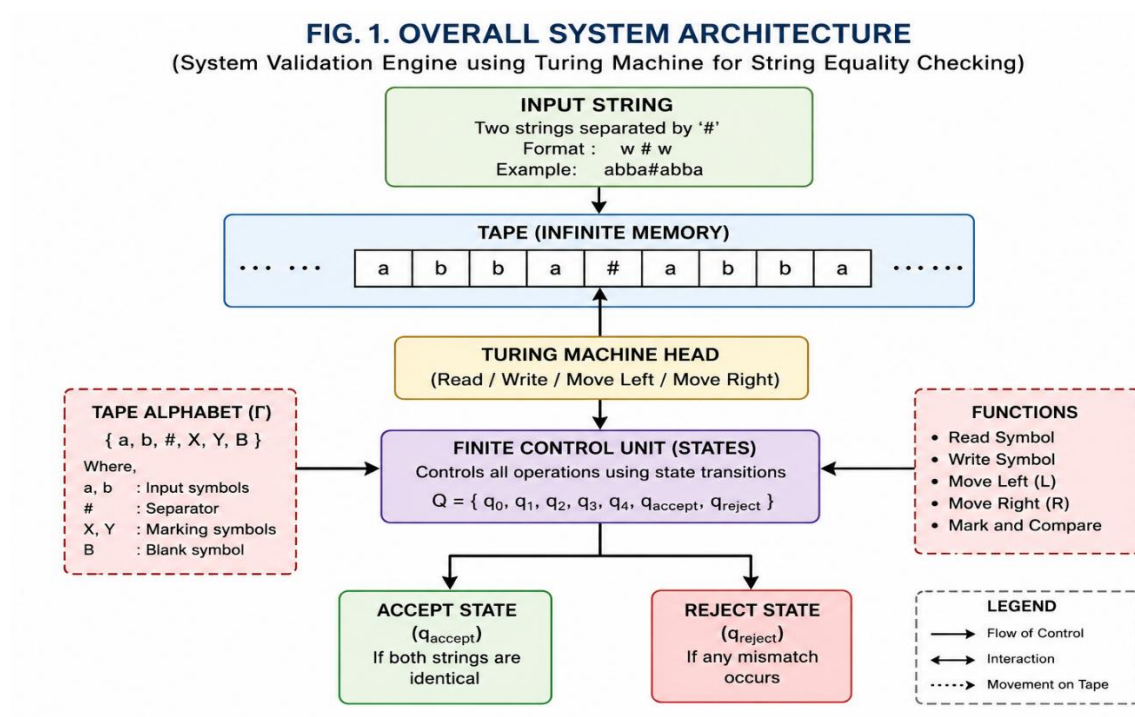


Figure 1 represents the architecture of the System Validation Engine based on a Turing Machine model. The system consists of the following components:

- Input String Tape containing data in the form **w#w**
- Turing Machine Head for reading, writing, and moving across the tape
- Finite Control Unit (State Transition System)
- Tape Alphabet including input symbols and marking symbols (X, Y)
- Decision Module (Accept / Reject State)

The system operates by reading the input string, marking matched symbols, and comparing corresponding characters on both sides of the separator “#”. The process continues until all symbols are verified. If all symbols match, the system enters the **Accept State**; otherwise, it transitions to the **Reject State**.

2. System Components

The architecture consists of the following components:

- **Input Tape:** Contains input in the form **w#w**
- **Tape Head:** Reads and writes symbols and moves left/right
- **Finite Control Unit:** Controls state transitions
- **Tape Alphabet:** {0, 1, #, X, Y, □}
- **Decision Module:** Accept or Reject state

Working Principle

- The input string is first written on the tape in the form **w#w**.
- The Turing Machine head scans the tape from left to right.
- The finite control unit decides transitions based on the current symbol.
- Matching symbols are marked to avoid reuse.
- The machine continues until all symbols are processed.
- If successful → system enters **Accept State**.
- If mismatch occurs → system enters **Reject State**.

4. Formal Definition of Turing Machine

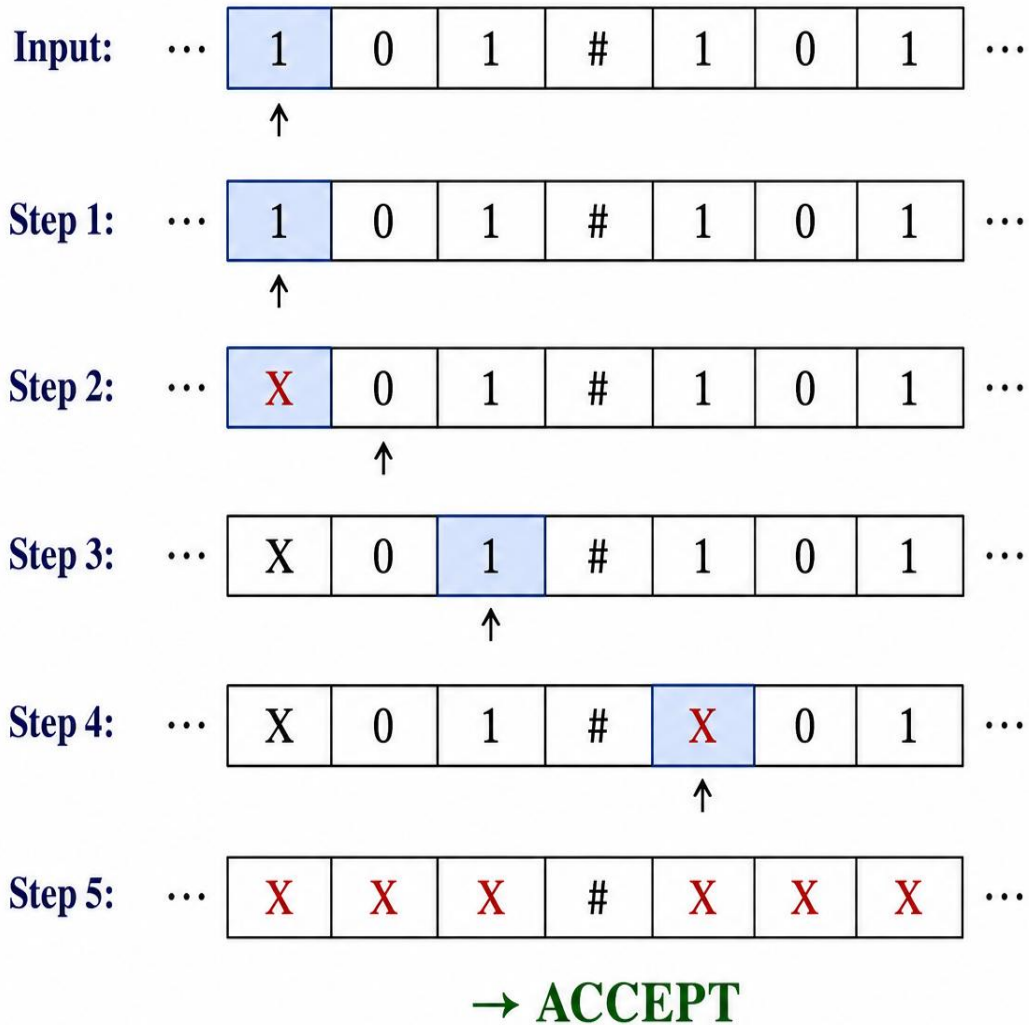
You should include this (VERY IMPORTANT for marks):

- $Q = \{q_0, q_1, q_2, q_3, q_{\text{accept}}, q_{\text{reject}}\}$
- $\Sigma = \{0, 1, \#\}$
- Tape alphabet = $\{0, 1, \#, X, Y, \square\}$
- Start state = q_0
- Accept state = q_{accept}
- Reject state = q_{reject}

5. Transition Function

State	Read	Write	Move	Next State
q_0	0	X	R	q_1
q_1	0	0	R	q_2
q_2	#	#	R	q_3
q_3	0	Y	L	q_4
q_4	X	X	R	q_0

6. Tape Transitions



7. Conclusion

The System Validation Engine demonstrates a clear and practical application of Turing Machine theory for solving the string equality problem. By systematically marking and comparing corresponding symbols on both sides of the separator “#”, the machine ensures that every character is verified accurately. Any mismatch or imbalance in length immediately leads to rejection, while complete successful matching results in acceptance.

This approach highlights the power of formal automata in performing structured computation and validation tasks. It not only reinforces theoretical concepts from automata theory but also shows their relevance in real-world computing problems such as data verification, parsing, and protocol validation. The model effectively bridges theory and practice, illustrating how complex logical decisions can be reduced to simple, rule-based state transitions.

REFERENCES

- [1] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Pearson Education, 3rd Edition.
- [2] P. Linz, *An Introduction to Formal Languages and Automata*, Jones & Bartlett Learning, 6th Edition.
- [3] M. Sipser, *Introduction to the Theory of Computation*, Cengage Learning, 3rd Edition.
- [4] J. C. Martin, *Introduction to Languages and the Theory of Computation*, McGraw-Hill Education, 4th Edition.
- [5] Department of Computer Science and Engineering, CS602PC Formal Languages and Automata Theory Lecture Notes, MR-22 Regulations.